深度学习
第 9 章　循环神经网络（RNN）

——当神经网络需要”记住”过去，该怎么做？

2026 年 3 月 16 日

本章题目导览：你将挑战哪些核心问题？

CNN 擅长处理静态图像，但现实中还有大量**顺序有关**的数据：你说的每一句话、写的每一行代码、股票的每一天价格……这些都是”时序数据”，顺序一变，意思就变。本章 6 道题从零打通 RNN 的核心脉络：

- **Q9-01（辨析填空）**：NLP 三种方法横向对比——同义词词典 vs 基于计数 vs 基于推理，各自解决什么问题、有何缺陷？
- **Q9-02（概念理解）**：词嵌入 vs 独热向量——为什么独热向量表示词语很”蠢”？词嵌入怎么就聪明了？
- **Q9-03（计算题）**：RNN 隐藏状态手动推导——给定权重和当前输入，一步步算出 h_t，把公式变成具体数字。
- **Q9-04（对比分析）**：RNN vs 前馈网络——权重共享、时间展开，RNN 到底和普通神经网络有什么本质区别？
- **Q9-05（代码阅读）**：古诗生成模型代码追踪——`PoetryRNN` 的数据流形状逐层追踪，损失函数为何要 `transpose`？
- **Q9-06（开放分析）**：文本生成中的”温度”采样——为什么用 `multinomial` 而不用 `argmax`？温度参数如何控制创作风格？

1　第 9 章　循环神经网络（RNN）与自然语言处理

在学 CNN 的时候，我们处理的都是”静态的”东西——一张图片，像素全部同时摆在那里，没有顺序关系。

但今天要处理的东西不一样了：

- ”我喜欢你”和”你喜欢我”——词完全一样，顺序不同，意思天差地别；
- 明天的气温，取决于今天、昨天、前天……
- 下一个字是什么，取决于你前面已经打了什么。

这类数据有一个共同特点：**顺序很重要，上下文很重要**。处理它们，就需要 RNN。

先建立一个总感觉：RNN 是什么

把 RNN 想象成一个边看书边记笔记的人。

- 普通神经网络（CNN、全连接）= 每次只看当前这一张卡片，上一张忘得干干净净；
- RNN = 每读到一个新词，会翻一眼自己之前的笔记，然后把新信息和旧笔记合并，更新成一份新笔记，传给下一步继续用。

RNN 最核心的能力就是四个字：**带着记忆往前走**。

Q9-01 NLP 三种核心方法横向对比

题目描述：

文档第 9.1 节介绍了计算机理解自然语言的三种方法：基于同义词词典、基于计数、基于推理。在做题之前，先来认识一下这三种思路的本质区别。

问题：

(1) 填写下表，将三种方法的关键特征补全：

方法	核心思想	主要缺点	是否自动化
基于同义词词典	人工构建词语网络，计算词语_____	_____（写出 2 条）	__
基于计数	统计词频，用_____（矩阵名称）表示词向量	对所有词_____极高	__
基于推理	根据_____预测当前位置的词	需要大量数据和计算资源	__

(2) ”语料库”（corpus）是什么？基于计数的方法为什么需要它？

(3) 基于推理的方法和基于计数的方法，在”如何获得词向量”这件事上，根本区别是什么？

来源溯源：[文档第 9 章 9.1 自然语言处理概述]

小白必读：为什么计算机理解语言这么难？

- **编程语言 vs 自然语言**：编程语言语法严格，一个符号只有一个含义，计算机按规则解析即可。但汉语英语是”软语言”——同一个词在不同语境下含义可能完全不同（”苹果”可以是水果，也可以是手机品牌），新词不断涌现，规则无法穷举。
- **计算机只认数字**：不管哪种方法，最终目的都是把”词”变成”向量（数字列表）”，让计算机能够进行数学运算——计算两个词有多相似、预测下一个词是什么。
- **三种方法的进化逻辑**：词典方法靠人工 → 计数方法靠统计 → 推理方法靠神经网络。越往后，越自动化，越能处理大规模数据，但也越需要算力。

生活比喻：三种”认识一个词”的方式

假设你是一个刚学中文的外国人，想搞懂”秋”是什么意思：

- **词典法**：翻字典，发现”秋”= 秋天、金秋、丰收……手工查好每个词的关系。但新词没收录怎么办？字典要一直更新，太累了。
- **计数法**：读了一万篇文章，发现”秋”总是和”风”” 叶”” 凉”” 稻”一起出现——靠这些邻居来猜它的意思。但要数遍所有文章里所有词两两共现，计算量爆炸。
- **推理法**：让神经网络做填空题：”金风玉露一相__”——靠这种上下文预测任务，网络自然就学会了每个词的”语义”，也就是词向量。这是现代 NLP 的核心思路。

参考答案与详细解析

(1) 填表：

方法	核心思想	主要缺点	是否自动化
基于同义词词典	人工构建词语网络，计算词语**相似度**	(1) 需要人工逐个定义，费时费力；(2) 新词维护成本极高；(3) 表现力有限	**否**
基于计数	统计词频，用**共现矩阵**表示词向量	对所有词向量化的**计算复杂度**极高	**是**
基于推理	根据**上下文**预测当前位置的词	需要大量数据和计算资源	**是**

(2) 语料库（corpus）的定义与作用：

语料库是**大量的文本数据集合**。基于计数的方法需要统计词频——即每个词和哪些词经常出现在一起（共现关系），这个统计必须建立在海量真实文本的基础上。语料库越大，统计出的词向量就越能反映真实的语言规律。没有语料库，就没有可统计的数据。

(3) 计数 vs 推理：获得词向量的根本区别：

核心对比

- **基于计数**：先统计整个语料库中的词频共现，**一次性算出**所有词的向量（共现矩阵的某列/某行）——这是统计方法，**不需要训练模型**。
- **基于推理**：设计一个神经网络，让它反复做”根据上下文猜词”的任务，**在训练过程中**逐渐学会词的向量表示——这是学习方法，**需要迭代优化模型**。

推理方法的词向量（也就是词嵌入）是训练神经网络时”顺带学出来”的副产品，同时也是 NLP 模型的第一层输入。Word2Vec、GloVe 这些著名的词向量技术，本质上都是基于推理思想的变体。

现实应用场景

词向量让搜索引擎”懂”你的意思：

早期搜索引擎只做关键词匹配——你搜”苹果手机”，它只返回包含这两个字的页面。基于词嵌入的现代搜索引擎能理解语义：搜”苹果手机”，它知道”iPhone”和这四个字语义相近，也会返回相关结果。这背后，就是训练好的词嵌入模型把”苹果手机”和”iPhone”映射到了向量空间中相近的位置。Google 的 BERT、百度的 ERNIE 等大语言模型，本质上都在做更深层的”理解语义”工作，其基础正是词嵌入。

Q9-02 词嵌入 vs 独热向量：谁更聪明？

题目描述：

文档 9.2.1 节提到，独热向量（One-Hot Vector）虽然容易构建，但存在两个致命缺陷，因此现代 NLP 普遍使用词嵌入（Word Embedding）来表示词语。

背景知识：

假设词表共有 $V = 5$ 个词：{我，喜欢，苹果，香蕉，手机}。

独热向量表示如下（每个词只有一个位置为 1，其余全为 0）：

$$\text{我} = [1, 0, 0, 0, 0], \quad \text{喜欢} = [0, 1, 0, 0, 0], \quad \text{苹果} = [0, 0, 1, 0, 0]$$

$$\text{香蕉} = [0, 0, 0, 1, 0], \quad \text{手机} = [0, 0, 0, 0, 1]$$

词嵌入则是：把每个词映射到一个低维稠密向量（如 3 维），这些向量由训练得到：

$$\text{苹果} \approx [0.8, 0.2, 0.1], \quad \text{香蕉} \approx [0.7, 0.3, 0.0], \quad \text{手机} \approx [0.1, 0.9, 0.8]$$

问题：

(1) 计算”苹果”和”香蕉”的独热向量余弦相似度；再计算”苹果”和”手机”的独热向量余弦相似度。结果说明了什么问题？

(2) 若词表大小为 $V = 50000$（这是一个正常的中等词表大小），独热向量每个需要多少维？相比之下，如果用 $d = 256$ 维的词嵌入，存储量之比大约是多少？

(3) 文档中说词嵌入矩阵（`nn.Embedding`）的参数形状是 $(\texttt{num_embeddings}, \texttt{embedding_dim})$，它在神经网络中是如何工作的？用”查表”比喻解释。

来源溯源：[文档第 9 章 9.2.1 什么是词嵌入]

小白必读：余弦相似度是什么？

余弦相似度是衡量两个向量”方向有多接近”的指标，公式为：

$$\cos(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}||\vec{b}|}$$

- 结果范围：$[-1, 1]$，越接近 1 说明越相似，等于 0 说明完全无关，等于 -1 说明完全相反。
- **对独热向量**：任意两个不同词的独热向量点积为 0（因为没有同为 1 的位置），所以余弦相似度恒为 0——”苹果”和”香蕉”跟”苹果”和”手机”，在独热空间里**一样不相关**，完全无法区分”语义近”和”语义远”。

生活比喻：独热向量是穿着囚服的词，词嵌入是有身份证的词

- **独热向量（囚服）**：每个词穿着一件只有自己编号的囚服，外表完全相同（都是一串 0 里有一个 1），除了编号外传达不了任何信息。问”苹果和香蕉像不像”？回答：一个是 3 号，一个是 4 号，一样的囚服，没区别。
- **词嵌入（身份证）**：每个词有一张丰富的身份证，上面写着它的各种”属性数值”（颜色倾向、可食用程度、科技感……）。苹果和香蕉的身份证数字很接近（都是水果），而苹果和手机的数字差很远（一个是食物，一个是电子产品）。
- **学习过程**：这张”身份证”不是人工填写的，而是神经网络在大量文本中自动学到的——它发现苹果和香蕉总在相似语境里出现，就自动给它们分配了相近的向量。

参考答案与详细解析

(1) 独热向量的余弦相似度：

”苹果” $= [0, 0, 1, 0, 0]$，”香蕉” $= [0, 0, 0, 1, 0]$，”手机” $= [0, 0, 0, 0, 1]$。

任意两个不同词的独热向量点积：

$$\text{苹果} \cdot \text{香蕉} = 0 \times 0 + 0 \times 0 + 1 \times 0 + 0 \times 1 + 0 \times 0 = 0$$

$$\text{苹果} \cdot \text{手机} = 0$$

所有独热向量模长均为 1，因此：

$$\cos(\text{苹果}, \text{香蕉}) = \frac{0}{1 \times 1} = 0 \quad \cos(\text{苹果}, \text{手机}) = 0$$

结论：苹果和香蕉（语义相近，都是水果）与苹果和手机（语义相远）的相似度**完全相同，都是 0**。独热向量无法区分语义近远，完全丢失了词语之间的语义关系。

(2) 存储量对比：

独热向量维度 $= V = 50000$，每个词占 50000 个数。

词嵌入维度 $= d = 256$，每个词只占 256 个数。

存储量之比：

$$\frac{50000}{256} \approx 195 \text{ 倍}$$

词嵌入的总参数量 $= V \times d = 50000 \times 256 = 1280$ 万个，虽然看起来多，但**每个词只用 256 个数表示**，比独热向量的 50000 维大幅压缩，且包含了语义信息。

(3) nn.Embedding 的工作原理——查表：

词嵌入矩阵形状为 (V, d)，可以理解成一本**"词向量字典"**：第 i 行就是第 i 个词的 d 维向量。

工作流程：三步查表

1. **分词 + 建词表**：把文本分成词，给每个词分配一个整数 ID（"我"=0，"苹果"=2……）
2. **查表**：把词的 ID 当作行索引，直接取出嵌入矩阵对应的那一行向量。`embed(torch.tensor(2))` 就是取出第 2 行，也就是"苹果"的词向量。
3. **梯度更新**：这个矩阵的每一行都是可学习参数。训练过程中，根据损失反向传播，不断调整每一行的数值，让语义相近的词向量越来越靠近。

```python
import torch, torch.nn as nn

# 词表大小5，词向量维度3
embed = nn.Embedding(num_embeddings=5, embedding_dim=3)

# "苹果"的 ID 是 2，直接取出第2行
apple_vec = embed(torch.tensor(2))
print(apple_vec)  # 输出 3 维向量，初始是随机值，训练后有语义

# 一次查多个词
batch_ids = torch.tensor([0, 2, 4])  # 我、苹果、手机
batch_vecs = embed(batch_ids)
print(batch_vecs.shape)  # torch.Size([3, 3])

```

现实应用场景

Word2Vec：第一个让词有"方向感"的词嵌入：

2013 年 Google 发布的 Word2Vec，通过在大规模语料上训练"预测上下文"任务，学出了惊人的词向量性质：$\vec{\text{国王}} - \vec{\text{男}} + \vec{\text{女}} \approx \vec{\text{女王}}$。这说明词向量空间里自发出现了"性别方向"和"王室方向"这样的语义轴。现代大语言模型（GPT、BERT）的 embedding 层依然继承了这一思想，只是用 Transformer 代替了简单的神经网络来训练更丰富的词语表示。

Q9-03 RNN 隐藏状态手动推导

题目描述：

RNN 层的核心公式是：

$$h_t = \tanh(h_{t-1}W_h + x_tW_x + b) \tag{1}$$

其中 x_t 是当前输入，h_{t-1} 是上一时刻的隐藏状态，W_x、W_h 是权重矩阵，b 是偏置。

注意：文档中标记符号为 W_{xh} 对应 W_x，W_{hh} 对应 W_h。

具体数值：

已知某 RNN 层中（tanh 激活**之前**的线性部分）：

$$x_t = \begin{bmatrix} 2.0 \\ -1.0 \end{bmatrix}, \quad h_{t-1} = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}, \quad W_x = \begin{bmatrix} 0.5 & -0.5 \\ 0.5 & 0.5 \end{bmatrix}, \quad W_h = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix}, \quad b = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

问题：

(1) 计算线性组合 $a_t = h_{t-1}W_h + x_tW_x + b$，写出完整推导过程。

(2) 在 a_t 上应用 tanh 激活函数，得出最终隐藏状态 h_t。
（提示：$\tanh(1.5) \approx 0.905$，$\tanh(-0.5) \approx -0.462$）

(3) 如果这是一首古诗的第一个字（$t = 1$），隐藏状态 h_0 应该初始化为多少？代码里是如何传入初始隐藏状态的？

来源溯源：[文档第 9 章 9.3.1 RNN 层介绍]

小白必读：h_t 的两个”来源”是什么？

每一步 RNN 的输入有两个：

- **当前词** x_t：这一时刻刚刚输入的信息（比如古诗第 t 个字的词向量）；
- **历史记忆** h_{t-1}：上一时刻积累下来的”状态摘要”，包含了前面所有时刻的信息痕迹。

这两者分别经过各自的权重矩阵”加工”，再合在一起，过一次 tanh 压缩，就得到了新的记忆 h_t。

为什么用 tanh 而不用 ReLU？

tanh 的输出范围是 $(-1, 1)$。RNN 会把 h_{t-1} 一次次地乘以 W_h 往前传，如果不压缩，数值会越乘越大（或越乘越小），导致”梯度爆炸”或”梯度消失”。tanh 把每一步的输出都限制在 $(-1, 1)$ 之间，起到”稳压器”的作用。

生活比喻：每一步都在”消化新闻 + 更新认知”

把 RNN 的每一步想象成一个人在看新闻滚动条：

- h_{t-1}（旧认知）：他到上一条新闻为止，脑子里对世界的整体判断；
- x_t（当前新闻）：刚收到的这一条新消息；
- W_h（旧认知的权重）：他有多信任自己以往的判断；
- W_x（新消息的权重）：他有多重视当前这条新消息；
- tanh（情绪稳定器）：不管收到多夸张的新闻，他的”认知更新量”都被控制在合理范围内，不会暴跳如雷（梯度爆炸）也不会完全无感（梯度消失）。

参考答案与详细解析

(1) 计算线性组合 a_t：

由于 $h_{t-1} = [0, 0]^\top$，所以历史记忆这一项贡献为零：

$$h_{t-1}W_h = \begin{bmatrix} 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \end{bmatrix}$$

当前输入的贡献（注意 x_t 是列向量，与 W_x 相乘为行向量，或按文档约定写为行乘矩阵）：
这里我们采用行向量左乘矩阵的约定，$x_t^\top = [2.0, -1.0]$：

$$x_t^\top W_x = \begin{bmatrix} 2.0 & -1.0 \end{bmatrix} \begin{bmatrix} 0.5 & -0.5 \\ 0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} 2.0 \times 0.5 + (-1.0) \times 0.5 \\ 2.0 \times (-0.5) + (-1.0) \times 0.5 \end{bmatrix}^\top = \begin{bmatrix} 1.5 & -1.5 \end{bmatrix}$$

等等，让我们重新按标准列向量约定来算（矩阵左乘列向量）：

$$W_x \cdot x_t = \begin{bmatrix} 0.5 & -0.5 \\ 0.5 & 0.5 \end{bmatrix} \begin{bmatrix} 2.0 \\ -1.0 \end{bmatrix} = \begin{bmatrix} 0.5 \times 2.0 + (-0.5) \times (-1.0) \\ 0.5 \times 2.0 + 0.5 \times (-1.0) \end{bmatrix} = \begin{bmatrix} 1.0 + 0.5 \\ 1.0 - 0.5 \end{bmatrix} = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}$$

加上偏置 $b = [0, 0]^\top$，最终：

$$\boxed{a_t = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}}$$

(2) 应用 tanh 激活，得到 h_t：

$$h_t = \tanh(a_t) = \begin{bmatrix} \tanh(1.5) \\ \tanh(0.5) \end{bmatrix} \approx \begin{bmatrix} 0.905 \\ 0.462 \end{bmatrix}$$

$$\boxed{h_t \approx \begin{bmatrix} 0.905 \\ 0.462 \end{bmatrix}}$$

直觉解读：第一步（$t = 1$）只有当前词的贡献，没有历史记忆。h_1 完全由第一个词决定，之后 h_1 会作为 h_{t-1} 传给第二步，把第一个词的信息带进去。

(3) 初始隐藏状态 h_0 的初始化：

通常将 h_0 初始化为**全零向量**，表示”序列开始前没有任何历史记忆”。

在古诗生成代码中，对应的写法是 `hidden = None`，PyTorch 的 `nn.RNN` 接收到 `hx=None` 时会自动将初始状态设为全零：

```
# 古诗生成代码中的初始化片段（文档 9.4.5）
hidden = None  # 初始化隐状态为 None，等价于全零

# 第一步推理
input = torch.LongTensor([[start_token]]).to(device)
output, hidden = model(input, hidden)
# 此后 hidden 被更新，下一步可直接传入
output, hidden = model(next_input, hidden)

```

形状提示：RNN API 的输入输出张量形状

- `input`：形状 [seq_len, batch_size, input_size]
- `hx`（初始隐状态）：形状 [num_layers, batch_size, hidden_size]
- `output`：形状 [seq_len, batch_size, hidden_size]
- `hn`（最终隐状态）：形状 [num_layers, batch_size, hidden_size]

Q9-04 RNN 与前馈网络的本质区别

题目描述：

文档 9.3.1 节指出，前馈网络的信号传播是**单向**的，而 RNN 的信号可以在层内**循环**。这两种架构在处理序列数据时有本质的区别。

问题：

(1) 判断题：下列说法正确吗？请逐条判断并给出理由。

a. "RNN 的 W_h 在处理序列第 1 个词和第 20 个词时是两套不同的权重矩阵。"

b. " 前馈网络也能处理文本，只需要把句子所有词的独热向量拼起来作为输入即可。"

c. "RNN 的输出 h_t 只包含当前时刻 x_t 的信息，不包含历史信息。"

(2) 文档说" 将神经网络的层数加深，可以更有效地提取层次信息"。在 `nn.RNN` 中，参数 `num_layers=2` 代表什么？第一个隐藏层的输出如何传给第二个隐藏层？

(3) 用下面的 PyTorch 代码，解释为什么 `output.shape` 是 `[1, 3, 16]` 而不是 `[1, 3, 8]`（8 是输入维度）？

```python
import torch
rnn = torch.nn.RNN(input_size=8, hidden_size=16, num_layers=2)
input = torch.rand(1, 3, 8)
hx = torch.randn(2, 3, 16)
output, hn = rnn(input=input, hx=hx)
print(output.shape)    # torch.Size([1, 3, 16])
print(hn.shape)        # torch.Size([2, 3, 16])

```

来源溯源：[文档第 9 章 9.3 循环网络层]

小白必读：什么是" 时间展开"？为什么说 RNN 有环路？

- **环路的意思**：RNN 单元的输出 h_t 会被" 循环" 送回到自身，作为下一个时刻的输入之一。这就是那个环路——$h_{t-1} \to \text{RNN} \to h_t \to \text{RNN} \to h_{t+1} \to \cdots$
- **时间展开**：处理一个长度为 T 的序列时，你可以把这个 RNN 单元" 展开" 成 T 个相连的副本，每个副本处理一个时刻，相邻副本之间通过 h_t 传递信息。展开后看起来就像一个很深的前馈网络，但**所有副本共享同一套权重**。
- **权重共享 = 不管序列多长，参数总量固定**：不管句子有 10 个词还是 100 个词，RNN 的参数量不变，因为同一套权重重复使用了 10 次或 100 次。这和全连接网络完全不同——全连接网络如果输入从 10 维变成 100 维，参数量会乘以 10。

参考答案与详细解析

(1) 判断题逐条解析：

a. **错误**。RNN 的核心特点正是**权重共享**——W_h、W_x、b 在处理整个序列的每一个时间步时**完全相同**。序列的不同位置用的是同一套权重，只是传入的 h_{t-1} 不同（携带了不同的历史信息），所以输出也不同。这正是 RNN 能以固定参数量处理任意长度序列的原因。

b. **有一定道理，但有严重缺陷**。把所有词拼起来确实可以输入全连接层，但存在两大问题：(1) 如果句子长度变化，输入维度就变化，全连接层无法处理不定长输入；(2) 输入维度过大（一个词独热向量 V 维，20 个词就是 $20V$ 维），参数量爆炸；(3) **顺序信息被丢弃**——拼接本身没有记录哪个词在前哪个词在后（当然可以靠位置编码补救，但这已经是 Transformer 的思路了）。

c. **错误**。h_t 不只包含当前时刻的信息，还包含**所有历史时刻的信息**（通过 h_{t-1} 一层层积累传递下来）。这正是 RNN 的核心价值所在——h_t 是一个" 历史摘要 + 当前输入" 的综合状态。

(2) `num_layers=2` 的含义：

`num_layers=2` 表示叠加了两层 RNN。第一层以原始输入序列 x_t 为输入，输出隐藏序列 $h_t^{(1)}$；第二层以第一层的输出 $h_t^{(1)}$ 为输入，输出最终隐藏序列 $h_t^{(2)}$。**每一层都有独立的** W_x、W_h、b。最终的 `output` 是第二层（最后一层）的输出，`hn` 是每一层最后时刻的隐藏状态。

(3) 为什么 `output.shape = [1, 3, 16]` 而不是 `[1, 3, 8]`：

三个维度的含义

- 第一维 `1`：序列长度 `seq_len=1`（只处理了 1 个时刻的输入）
- 第二维 `3`：批量大小 `batch_size=3`（同时处理 3 个样本）
- 第三维 `16`：**隐藏层大小 `hidden_size=16`**，而不是输入大小 `input_size=8`

输出维度是 `hidden_size`，原因：RNN 的输出就是每个时刻的隐藏状态 h_t，而 h_t 的维度是 `hidden_size`，和输入维度 `input_size` 无关。就像全连接层 `Linear(8, 16)` 的输出是 16 维，不是 8 维——输出维度由网络结构决定，不由输入维度决定。

现实应用场景

多层 RNN 在语音识别中的应用：

深层双向 RNN（Bidirectional RNN）是 2014 年前语音识别的主流架构。第一层 RNN 学习声学特征的低级时序模式（音素级别），第二层学习更高级的语言结构（音节、词级别）。Google 的早期语音识别系统 DeepSpeech 就用了多层 RNN。不过从 2020 年起，Transformer 架构逐渐在语音识别中取代 RNN，主要原因是 Transformer 可以**并行训练**（RNN 必须逐步计算，无法并行），训练速度快得多。

Q9-05 古诗生成代码追踪：数据流形状分析

题目描述：

阅读文档 9.4 节的古诗生成案例代码，回答以下问题。

```python
class PoetryRNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim=128, hidden_size=256,
                 num_layers=1):
        super().__init__()
        self.embed = nn.Embedding(num_embeddings=vocab_size,
                                  embedding_dim=embedding_dim)
        self.rnn = nn.RNN(input_size=embedding_dim,
                          hidden_size=hidden_size, num_layers=num_layers)
        self.linear = nn.Linear(in_features=hidden_size,
                                out_features=vocab_size)

    def forward(self, input, hx=None):
        embed = self.embed(input)        # 第1步
        output, hidden = self.rnn(embed, hx)    # 第2步
        output = self.linear(output)     # 第3步
        return output, hidden

model = PoetryRNN(len(vocab), 256, 512, 2).to(device)

```

训练时，输入 x 的形状为 [batch_size=32, seq_len=24]（32 首诗，每首取 24 个字）。

问题：

(1) 假设 vocab_size=3000，按照模型构造参数（embedding_dim=256, hidden_size=512, num_layers=2），逐步追踪以下四个张量的形状：

a. x（模型输入）：_____

b. embed（词嵌入层输出）：_____

c. output（RNN 层输出）：_____

d. output（线性层输出）：_____

(2) 训练代码中有这样一行：

```python
loss_value = loss(output.transpose(1, 2), y)

```

为什么要 transpose(1, 2)？CrossEntropyLoss 期望的输入形状是什么？

(3) 训练时的目标序列 y 和输入序列 x 的关系是什么？为什么要这样设计？

来源溯源：[文档第 9 章 9.4.2 自定义 Dataset / 9.4.3 搭建模型 / 9.4.4 模型训练]

小白必读：RNN 为什么要先过词嵌入再过 RNN？

- **输入流水线**：文字 → 整数 ID → 词嵌入向量 → RNN → 预测下一个词的概率。
- **为什么不能直接把整数 ID 送进 RNN？**整数 ID（比如"月"=1234，"花"=567）是人工分配的编号，没有任何语义含义，1234 不比 567 更"月亮"。把这些编号直接送进 RNN，模型会把 ID 值当成数量关系来处理，这毫无意义。词嵌入的作用就是把无意义的整数变成有语义含义的浮点向量。
- **为什么要 `nn.RNN` 之后再接 `nn.Linear`？**RNN 的输出 h_t 维度是 `hidden_size=512`，但我们最终要预测的是"下一个字是词表中哪个字"，输出应该是 `vocab_size=3000` 维的概率分布。`nn.Linear(512, 3000)` 就是把隐藏状态"解码"成词表上的 logits（未归一化概率）。

参考答案与详细解析

(1) 逐层形状追踪：

注意：`nn.RNN` 期望输入形状为 `[seq_len, batch_size, input_size]`，但 `x` 的形状是 `[batch_size, seq_len]`，需要经过词嵌入和必要的维度调整。我们先追踪各层直接的输出形状：

步骤	操作	输出形状	说明
输入	`x`	`[32, 24]`	32 首诗，每首 24 字（整数 ID）
第 1 步	`embed(x)`	`[32, 24, 256]`	每个字 ID 变成 256 维词向量
（转置）	调整维度喂给 RNN	`[24, 32, 256]`	seq_len 在前，batch 在中
第 2 步	`rnn(embed)`	`[24, 32, 512]`	每个时刻输出 512 维隐藏状态
第 3 步	`linear(output)`	`[24, 32, 3000]`	每个时刻预测 3000 个字的得分

(2) 为什么要 `transpose(1, 2)`：

`CrossEntropyLoss` 对多类别分类的期望输入形状为：

```
[batch_size, num_classes, ...]
```

即类别维度必须是**第二个维度**（维度索引 1）。

而 RNN 输出经过 `linear` 后形状是 `[seq_len=24, batch_size=32, vocab_size=3000]`，如果直接换成批次在前，则是 `[32, 24, 3000]`——此时类别维度（3000）在第三个位置（索引 2），不满足 `CrossEntropyLoss` 的要求。

`output.transpose(1, 2)` 交换第 1、2 维，把 `[32, 24, 3000]` 变成 `[32, 3000, 24]`，类别维度就到了索引 1，符合要求。

CrossEntropyLoss 期望的输入形状

- `input`（预测）：形状 `[N, C]` 或 `[N, C, d1, d2, ...]`，C 是类别数
- `target`（标签）：形状 `[N]` 或 `[N, d1, d2, ...]`，存储类别整数索引
- 本案例：`output.transpose(1,2)` 形状为 `[32, 3000, 24]`，`y` 形状为 `[32, 24]`，完全匹配

(3) 目标序列 `y` 和输入序列 `x` 的关系：

从 PoetryDataset 的代码可以看出：

```
self.data.append((seq[i : i + self.seq_len],        # x
                  seq[i + 1 : i + 1 + self.seq_len])) # y
```

y 比 x **整体向右偏移了 1 个字**——即 $y_t = x_{t+1}$。

这叫"下一个字预测"(Next Token Prediction)任务：模型在看到 x_t 时，要预测下一个字 $y_t = x_{t+1}$。训练时，已经知道了正确答案（y），通过交叉熵损失衡量预测有多准，再反向传播更新权重。这是语言模型的标准训练方式，GPT 系列大模型也是这样训练的，只是规模大了几千倍。

Q9-06 文本生成中的采样策略：argmax vs multinomial

题目描述：

文档 9.4.5 节的古诗生成代码中，生成下一个字的方式是：

```
prob = torch.softmax(output[0, 0], dim=-1)   # 计算概率分布
next_token = torch.multinomial(prob, 1)       # 从概率分布中随机采样
```

注意到这里用的是 multinomial（多项式采样），而不是 argmax（直接取概率最大的词）。

问题：

(1) 如果改用 argmax 策略，即每次都选概率最大的词，会产生什么问题？为什么古诗生成不适合这样做？

(2) torch.softmax 把 RNN 输出的 logits（原始得分）变成了概率分布，具体做了什么数学变换？如果词表里有 3 个词，logits 为 $[3.0, 1.0, 0.5]$，计算其 softmax 概率。（结果保留两位小数）

(3) 在文本生成中，有一种叫"温度（Temperature）"的参数，可以控制生成文本的"创意程度"：

$$\text{prob} = \text{softmax}\left(\frac{\text{logits}}{T}\right)$$

当 $T < 1$ 时，生成更保守（倾向于高概率词）；当 $T > 1$ 时，生成更随机（低概率词也有机会被选中）。请解释其背后的数学原因，并说明在古诗生成场景下，T 选大一点还是小一点更好。

来源溯源：[文档第 9 章 9.4.5 测试]

小白必读：Softmax 是什么？为什么必须先算 softmax 再采样？

- **Softmax 的作用**：把任意实数向量（logits）变成**各元素之和为 1 的概率向量**。公式为 $p_i = e^{z_i} / \sum_j e^{z_j}$。无论输入多大多小，输出都是 $[0, 1]$ 区间内的合法概率。
- **为什么要先 softmax 再采样？**torch.multinomial 需要概率分布作为输入，而 RNN 输出的 logits 可以是任意实数（可以是负数），不能直接当概率用。
- **argmax vs 采样的差别**：argmax 是确定性的，同一个模型在同样上下文下永远输出同样的词，一直重复。采样是随机的，带来多样性和"创意感"，非常适合诗歌这种需要变化的生成任务。

生活比喻：点菜时的”纠结程度”就是温度

你去餐厅点菜，菜单上有三道菜，你分别打了 $[3.0, 1.0, 0.5]$ 分：

- **argmax 策略（极度理性）**：每次都点评分最高的那道，从不尝试别的。时间长了，永远只吃那一道菜——模型也是如此，会产生极度单调重复的文本。
- **低温采样 $T = 0.5$（保守型食客）**：评分差距被放大，高分菜的优势更明显，你大概率还是点高分菜，但偶尔也会鼓起勇气试试第二名。
- **高温采样 $T = 2.0$（冒险型食客）**：评分差距被压缩，三道菜的概率趋于均匀，你可能随机点任何一道——模型生成的文本更有创意，但也可能产生”不知所云”的内容。
- **古诗生成的选择**：既要押韵、有章法（不能太随机），又要有一定新意（不能只会重复）。一般 $T \in [0.8, 1.2]$ 是较好的平衡点——稍微保守，但保留一定的随机性。

参考答案与详细解析

(1) argmax 策略的问题：

贪心搜索的致命缺陷——重复生成。如果每步都取概率最高的词，模型会陷入”循环陷阱”：比如生成到”春风吹又生”之后，下一步概率最高的词可能又是”春”，然后再是”风”……无限循环，无法生成有新意的内容。

更根本的问题是：**局部最优不等于全局最优**。每步选最可能的词，不代表整首诗最可能。就像下棋，每步都走局部最好的一步，不一定赢得了整局。

(2) Softmax 计算：

Softmax 公式：$p_i = \dfrac{e^{z_i}}{\sum_j e^{z_j}}$

给定 logits $= [3.0, 1.0, 0.5]$：

$$e^{3.0} \approx 20.09, \quad e^{1.0} \approx 2.718, \quad e^{0.5} \approx 1.649$$

$$\text{总和} = 20.09 + 2.718 + 1.649 = 24.457$$

$$p_1 = \frac{20.09}{24.457} \approx 0.82, \quad p_2 = \frac{2.718}{24.457} \approx 0.11, \quad p_3 = \frac{1.649}{24.457} \approx 0.07$$

$$\boxed{\text{softmax}([3.0, 1.0, 0.5]) \approx [0.82,\ 0.11,\ 0.07]}$$

三者之和 $= 1.00$，满足概率分布要求。第一个词被选中的概率高达 82%，体现了 softmax 对最大值的”放大”效果。

(3) 温度参数的数学原理：

当 $T < 1$ 时（如 $T = 0.5$）：等效于把 logits 放大（logits/0.5 $= 2 \times$ logits），原来差距 2 的两个值变成差距 4。Softmax 对差距更敏感，高分值的概率被进一步放大，分布更”尖锐”（峰值更高），模型更倾向于选最高概率的词。

当 $T > 1$ 时（如 $T = 2$）：等效于把 logits 缩小（logits/2），原来差距 2 的两个值变成差距 1。Softmax 对差距不敏感，概率分布趋于均匀，各个词都有机会被选中，文本多样性增加。

当 $T \to 0$：退化为 argmax（完全确定性）。

当 $T \to \infty$：退化为均匀随机选词（完全随机）。

古诗生成中温度的实用建议

- $T < 0.7$：输出较工整，但容易重复，缺乏新意
- $T = 0.8 \sim 1.0$：平衡模式，通常产生质量较好的古诗
- $T > 1.2$：创意较强但可能产生不押韵、无逻辑的输出
- **实践小技巧**：可以先用低温生成几个候选，再从中挑选最自然的一首

```
# 加入温度参数的生成代码（对原代码的改进版）
T = 0.9  # 温度参数，可以调节
prob = torch.softmax(output[0, 0] / T, dim=-1)  # 除以温度再 softmax
next_token = torch.multinomial(prob, 1)

```

现实应用场景

ChatGPT 的 temperature 参数——你每天都在用的采样策略：
ChatGPT API 中有一个 `temperature` 参数（默认 1.0，范围 0–2），本质上就是本题讨论的温度采样。写代码时建议 $T = 0.2$（更精确，避免随机错误）；写诗歌、故事时建议 $T = 1.2$（更有创意）；需要稳定可复现结果时设 $T = 0$（等价于 argmax）。OpenAI 还提供了 `top_p`（核采样）参数作为温度的替代方案——只从累积概率超过 p 的词中采样，也是控制多样性与准确性平衡的常用手段。

2 本章小结：把公式化成脑中的画面

学完这 6 道题，如果还觉得”概念飘在空中”，试试把整章内容浓缩成这几句画面：

- **NLP 三步进化**：人工词典（查字典）→ 统计共现（数邻居）→ 神经推理（做填空题）。
- **词嵌入**：把每个词变成一张”语义身份证”，训练完了语义相近的词向量也相近。
- **RNN 是什么**：边读序列边更新笔记，笔记（h_t）传给下一步继续用。
- **权重共享**：不管序列多长，始终用同一套规则处理每个时刻——这是 RNN 处理可变长序列的秘诀。
- **古诗生成的本质**：学会”已知前 t 个字，预测第 $t+1$ 个字”，然后把预测结果不断喂回模型，一字一字地生成。
- **采样 vs argmax**：argmax 太死板，采样带来多样性，温度控制创意程度。

给零基础读者的一句总比喻

如果把序列建模比作写一首歌：

- 前馈网络：只看当前这一拍，不知道前面唱了什么，完全不会作词；
- RNN：有一本随手记录的小本子，每写一个字都翻翻本子，但本子页数有限，写得太长了前面的字迹会模糊；
- 古诗生成模型：拿着一本装满唐诗的课本练习”根据上文续写下一个字”，练多了，隐约学会了格律和意境——虽然还达不到李白的水平，但偶尔也能写出”月落乌啼霜满天”这样的意境。